Guide complet — Déployer un projet Django + React avec Docker sur un VPS

Ce guide détaille une méthode claire et robuste pour mettre en production un projet avec :

- Frontend React/Vite
- Backend Django
- PostgreSQL
- Docker + Docker Compose
- Nginx
- HTTPS
- Protection temporaire par mot de passe

Il tient compte d'un cas réel où **plusieurs sites tournent déjà sur le même VPS**, donc on évite soigneusement les conflits de ports.

---

1. Objectif de l'architecture

Architecture recommandée :

- **Nginx** sert le frontend buildé (`frontend/dist`)
- **Nginx** proxy `/api/` vers Django
- **Docker** lance :
  - le backend Django
  - la base PostgreSQL du projet
- Le frontend **n'est pas lancé en mode dev** sur le VPS en production

Pourquoi cette architecture

Elle est plus simple et plus stable que :

- `npm run dev` en production
- exposer beaucoup de ports
- mélanger plusieurs applis sur les mêmes ports

---

2. Pré-requis

Avoir :

- un VPS Ubuntu
- un domaine pointé vers le VPS
- un projet sur GitHub
- un accès SSH
- un utilisateur avec `sudo`

---

3. Structure de projet attendue

Exemple :

```
basogol/
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── manage.py
│   ├── config/
│   └── .env.docker
├── frontend/
│   ├── package.json
│   ├── index.html
│   ├── src/
│   ├── dist/
│   └── .env.production
└── docker-compose.yml
```

---

4. Cloner le projet sur le VPS

Exemple de dossier final :

```
/var/www/projects/basogol
```

Clonage :

```
cd /var/www/projects
git clone https://github.com/TON_ORG/TON_REPO.git basogol
cd basogol
git checkout main
```

Mise à jour plus tard :

```
cd /var/www/projects/basogol
git fetch origin
git reset --hard origin/main
```

---

5. Installer les dépendances système

Nginx

```
sudo apt update
sudo apt install -y nginx
```

Docker (dépôt officiel)

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg

sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo   "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu   $(.
/etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" |   sudo
tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin

sudo systemctl enable docker
sudo systemctl start docker
```

Utiliser Docker sans `sudo`

```
sudo groupadd docker 2>/dev/null || true
sudo usermod -aG docker $USER
```

Puis **déconnexion / reconnexion SSH**.

Vérification :

```
groups
docker --version
docker compose version
```

---

6. Vérifier les ports déjà utilisés

Quand plusieurs projets tournent sur le même VPS, il faut d'abord vérifier les ports.

```
sudo ss -ltnp
ps aux | grep gunicorn
sudo lsof -i :8000
sudo lsof -i :5432
```

Exemple typique

- `eyele` sur `8000`
- une autre app sur `8001`
- PostgreSQL système sur `5432`

Choix conseillé pour le nouveau projet

Pour éviter les conflits :

- backend Django du projet : **8002**
- PostgreSQL Docker : **pas exposé sur l'hôte**

---

7. Fichier `backend/.env.docker`

Créer :

```
nano /var/www/projects/basogol/backend/.env.docker
```

Exemple :

```
SECRET_KEY=TA_VRAIE_CLE_DE_PROD
DEBUG=False

ALLOWED_HOSTS=basogolhive.com,www.basogolhive.com,127.0.0.1,localhost

DB_NAME=basogol
DB_USER=postgres
DB_PASSWORD=TON_MOT_DE_PASSE_DB
DB_HOST=db
DB_PORT=5432

EMAIL_HOST=ssl0.ovh.net
EMAIL_PORT=587
EMAIL_USE_TLS=True
EMAIL_HOST_USER=projects-global@basogolhive.com
EMAIL_HOST_PASSWORD=TON_VRAI_MOT_DE_PASSE_EMAIL
DEFAULT_FROM_EMAIL=projects-global@basogolhive.com

CONTACT_RECIPIENT_TECH=projects-tech@basogolhive.com
CONTACT_RECIPIENT_MARKETING=projects-studio@basogolhive.com
```

Important

- `DB_HOST=db` si le service PostgreSQL s'appelle `db` dans Compose
- `ALLOWED_HOSTS` doit contenir le domaine
- ajouter `127.0.0.1,localhost` aide pour certains tests via `curl`

---

8. Fichier `frontend/.env.production`

Créer :

```
nano /var/www/projects/basogol/frontend/.env.production
```

Contenu :

```
VITE_API_BASE=https://basogolhive.com
VITE_API_PREFIX=/api
```

---

9. `settings.py` Django — points essentiels

Exemple de logique saine :

```python
from pathlib import Path
import os
from dotenv import load_dotenv

BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")

SECRET_KEY = os.getenv("SECRET_KEY", "change-me")
DEBUG = os.getenv("DEBUG", "False").lower() == "true"

ALLOWED_HOSTS = [
    host.strip()
    for host in os.getenv("ALLOWED_HOSTS", "127.0.0.1,localhost").split(",")
    if host.strip()
]

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.getenv("DB_NAME", ""),
        "USER": os.getenv("DB_USER", ""),
        "PASSWORD": os.getenv("DB_PASSWORD", ""),
        "HOST": os.getenv("DB_HOST", "127.0.0.1"),
        "PORT": os.getenv("DB_PORT", "5432"),
    }
}

EMAIL_BACKEND = "django.core.mail.backends.smtp.EmailBackend"
EMAIL_HOST = os.getenv("EMAIL_HOST", "")
EMAIL_PORT = int(os.getenv("EMAIL_PORT", "587"))
EMAIL_USE_TLS = os.getenv("EMAIL_USE_TLS", "True").lower() == "true"
EMAIL_HOST_USER = os.getenv("EMAIL_HOST_USER", "")
EMAIL_HOST_PASSWORD = os.getenv("EMAIL_HOST_PASSWORD", "")
DEFAULT_FROM_EMAIL = os.getenv("DEFAULT_FROM_EMAIL", EMAIL_HOST_USER)
```

CORS / CSRF

```python
if DEBUG:
    CORS_ALLOWED_ORIGINS = [
        "http://localhost:5173",
        "http://127.0.0.1:5173",
        "http://localhost:5174",
        "http://127.0.0.1:5174",
    ]
    CSRF_TRUSTED_ORIGINS = [
        "http://localhost:5173",
        "http://127.0.0.1:5173",
        "http://localhost:5174",
        "http://127.0.0.1:5174",
    ]
```

```
else:
    CORS_ALLOWED_ORIGINS = [
        "https://basogolhive.com",
        "https://www.basogolhive.com",
    ]
    CSRF_TRUSTED_ORIGINS = [
        "https://basogolhive.com",
        "https://www.basogolhive.com",
    ]
```

---

10. `docker-compose.yml` recommandé

Version simple et propre : **backend + db uniquement**

```
services:
  db:
    image: postgres:16
    container_name: basogol_db
    restart: unless-stopped
    environment:
      POSTGRES_DB: basogol
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: TON_MOT_DE_PASSE_DB
    volumes:
      - postgres_data:/var/lib/postgresql/data

  backend:
    build:
      context: ./backend
    container_name: basogol_backend
    restart: unless-stopped
    command: sh -c "python manage.py migrate && python manage.py collectstatic --
noinput && python manage.py runserver 0.0.0.0:8000"
    volumes:
      - ./backend:/app
    ports:
      - "8002:8000"
    env_file:
      - ./backend/.env.docker
    depends_on:
      - db

volumes:
  postgres_data:
```

Pourquoi cette version

- pas de conflit PostgreSQL (`5432` non publié)
- backend exposé sur `8002`

- pas de frontend Docker inutile, puisque Nginx sert déjà `dist`

---

11. `backend/Dockerfile`

Exemple :

```
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

RUN apt-get update && apt-get install -y     gcc     libpq-dev     && rm -rf
/var/lib/apt/lists/*

COPY requirements.txt /app/
RUN pip install --no-cache-dir -r requirements.txt

COPY . /app/

EXPOSE 8000

CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

Remarque

Pour une vraie prod avancée, on préfère `gunicorn`, mais cette version est acceptable pour un déploiement initial.

---

12. Corriger `frontend/index.html`

Exemple correct :

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/png" href="/basogollogo.png?v=7" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Basogol-Hive</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Important

Ne pas laisser :

```
<title className="notranslate" ...>
```

`className` n'est pas du HTML standard.

---

13. Build du frontend

```
cd /var/www/projects/basogol/frontend
npm install
npm run build
```

Vérification :

```
ls dist
cat dist/index.html
```

Le build final doit être dans :

```
/var/www/projects/basogol/frontend/dist
```

---

14. Configuration Nginx

Créer / remplacer :

```
sudo nano /etc/nginx/sites-available/basogolhive.com
```

Contenu recommandé :

```
server {
    listen 80;
    server_name basogolhive.com www.basogolhive.com;

    return 301 https://$host$request_uri;
}

server {
```

```
    listen 443 ssl;
    server_name basogolhive.com www.basogolhive.com;

    root /var/www/projects/basogol/frontend/dist;
    index index.html;

    ssl_certificate /etc/letsencrypt/live/basogolhive.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/basogolhive.com/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;

    location / {
        try_files $uri /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8002/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /static/ {
        proxy_pass http://127.0.0.1:8002/static/;
        proxy_set_header Host $host;
    }

    location /media/ {
        proxy_pass http://127.0.0.1:8002/media/;
        proxy_set_header Host $host;
    }
}
```

Activer le site

```
sudo ln -sf /etc/nginx/sites-available/basogolhive.com /etc/nginx/sites-
enabled/basogolhive.com
```

Tester

```
sudo nginx -t
sudo systemctl reload nginx
```

---

15. Rendre le site privé temporairement

Installer :

```
sudo apt install -y apache2-utils
```

Créer le fichier d'auth :

```
sudo htpasswd -c /etc/nginx/.htpasswd_basogol ekwel
```

Ajouter dans le bloc HTTPS :

```
auth_basic "Basogol Private";
auth_basic_user_file /etc/nginx/.htpasswd_basogol;
```

Exemple complet :

```
server {
    listen 443 ssl;
    server_name basogolhive.com www.basogolhive.com;

    root /var/www/projects/basogol/frontend/dist;
    index index.html;

    ssl_certificate /etc/letsencrypt/live/basogolhive.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/basogolhive.com/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;

    auth_basic "Basogol Private";
    auth_basic_user_file /etc/nginx/.htpasswd_basogol;

    location / {
        try_files $uri /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8002/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /static/ {
        proxy_pass http://127.0.0.1:8002/static/;
        proxy_set_header Host $host;
    }

    location /media/ {
```

```
        proxy_pass http://127.0.0.1:8002/media/;
        proxy_set_header Host $host;
    }
}
```

Puis :

```
sudo nginx -t
sudo systemctl reload nginx
```

---

16. Lancer les conteneurs

```
cd /var/www/projects/basogol
docker compose down --remove-orphans
docker compose up -d --build
```

Vérification :

```
docker compose ps
docker compose logs --tail=100 backend
docker compose logs --tail=100 db
```

---

17. Vérifier PostgreSQL et les migrations

PostgreSQL du projet

Le service Docker `db` est correct si :

```
docker compose ps
```

affiche `basogol_db Up`.

Migrations

Comme le backend lance :

```
python manage.py migrate
```

au démarrage, elles sont normalement appliquées automatiquement.

Pour vérifier :

```
docker compose exec backend python manage.py showmigrations
docker compose exec backend python manage.py migrate --plan
```

Résultat attendu :

- `showmigrations` avec `[X]`
- `migrate --plan` sans plan restant

---

18. Tester la route API correctement

Mauvais test classique

```
curl -I http://127.0.0.1:8002/api/newsletter/
```

Si tu obtiens :

```
HTTP/1.1 400 Bad Request
```

cela ne veut pas dire **404**. Cela signifie juste :

- route trouvée
- requête invalide

Bon test

```
curl -X POST http://127.0.0.1:8002/api/newsletter/   -H "Host: basogolhive.com"
-H "Content-Type: application/json"   -d
'{"email":"tonmail@example.com","language":"fr"}'
```

Pourquoi `Host`

Django peut refuser les requêtes si `ALLOWED_HOSTS` n'autorise pas `127.0.0.1`.

---

19. Vérifier le domaine

Si le site est privé

Sans credentials :

```
curl -s https://basogolhive.com | head -20
```

retournera :

```
401 Authorization Required
```

C'est normal.

Voir le vrai HTML

```
curl -u ekwel:TON_MOT_DE_PASSE -s https://basogolhive.com | head -50
```

---

20. Google et l'ancien snippet

Même si le site live a changé, Google peut encore afficher un ancien extrait.

Important

Si le site est protégé par mot de passe, Google ne pourra pas mettre à jour rapidement l'index.

Donc :

- le site live peut être correct
- Google peut afficher un ancien snippet pendant un moment

---

21. Erreurs fréquentes

Port 8000 déjà utilisé

```
sudo lsof -i :8000
```

Solution :

- choisir `8002`

Port 5432 déjà utilisé

Ne pas publier :

```
ports:
```

```
    - "5432:5432"
```

Docker permission denied

Ajouter l'utilisateur au groupe docker, puis se reconnecter.

`location directive is not allowed here`

ou

`server directive is not allowed here`

Fichier Nginx mal structuré.

`conflicting server name`

Plusieurs blocs `server` pour le même domaine sur le même port.

`400 Bad Request`

Ce n'est pas forcément une route cassée. Ça peut être :

- méthode incorrecte
- requête vide
- `ALLOWED_HOSTS`

---

22. Mise à jour du projet

Quand tu modifies le code localement et pousses :

```
cd /var/www/projects/basogol
git fetch origin
git reset --hard origin/main

cd frontend
npm run build
cd ..

docker compose down --remove-orphans
docker compose up -d --build

sudo nginx -t
sudo systemctl reload nginx
```

---

23. Rendre le site public plus tard

Supprimer dans nginx :

```
auth_basic "Basogol Private";
auth_basic_user_file /etc/nginx/.htpasswd_basogol;
```

Puis :

```
sudo nginx -t
sudo systemctl reload nginx
```

---

24. Checklist finale

VPS

- [ ] Nginx installé
- [ ] Docker installé
- [ ] Domaine pointe vers le VPS

Projet

- [ ] Repo cloné dans `/var/www/projects/basogol`
- [ ] `backend/.env.docker` créé
- [ ] `frontend/.env.production` créé
- [ ] `docker-compose.yml` correct
- [ ] frontend buildé

Nginx

- [ ] `root /var/www/projects/basogol/frontend/dist`
- [ ] `/api/` vers `127.0.0.1:8002/api/`
- [ ] SSL actif
- [ ] `sudo nginx -t` OK

Docker

- [ ] `basogol_backend` Up
- [ ] `basogol_db` Up

Django

- [ ] migrations OK
- [ ] `/api/newsletter/` répond

---

25. Commandes essentielles

Lancer

```
cd /var/www/projects/basogol
docker compose down --remove-orphans
docker compose up -d --build
```

Vérifier

```
docker compose ps
docker compose logs --tail=100 backend
docker compose logs --tail=100 db
sudo nginx -t
```

Mettre à jour

```
git fetch origin
git reset --hard origin/main
cd frontend && npm run build && cd ..
docker compose up -d --build
sudo systemctl reload nginx
```

Tester backend

```
curl -X POST http://127.0.0.1:8002/api/newsletter/   -H "Host: basogolhive.com"
-H "Content-Type: application/json"   -d
'{"email":"tonmail@example.com","language":"fr"}'
```

Tester domaine privé

```
curl -u ekwel:TON_MOT_DE_PASSE -s https://basogolhive.com | head -50
```